



Цифровые технологии управления
транспортными процессами

Сервис-ориентированное
программирование

REST

REST API like a pro



Вы умеете писать это:

```
@RestController
@RequestMapping("/authors")
public class AuthorController {

    @GetMapping("/{id}")
    public Author getAuthor(@PathVariable Long
id) {
        return
repository.findById(id).orElseThrow();
    }
}
```



```
{ "id": 1, "firstName": "Лев", "lastName":
"Толстой" }
```



Это работает. Но все пойдет не так...

- Поле переименовали
 - Бекенд переименовал name → fullName. Фронтенд упал в продакшене. Никто не знал что это сломает что-то ещё - не было контракта.
- PUT вместо PATCH
 - Мобильное приложение отправило PATCH { "bio": "новая биография" }. Сервер ожидал все поля (PUT-семантика).
 - email, phone, birthDate - затёрлись нулями.
- Клиент жёстко зашил URL
 - Сервис переехал с /api/v1/authors на /api/v2/users. 250 мест в клиентском коде пришлось обновлять вручную.
- Проблема не в коде. Проблема в отсутствии договорённостей.



API-контракт

- API-контракт - явное соглашение между сервером и клиентом:
 - какие URL существуют
 - какие данные принимаются и возвращаются
 - какие ошибки возможны
- Контракт как Maven-зависимость - если сервер нарушил контракт - проект не собирается. Точка.



Структура проекта





Maven Reactor

```
<groupId>edu.rutmit.demo</groupId>
<artifactId>demo-rest-root</artifactId>
<packaging>pom</packaging>

<modules>
  <module>books-api-contract</module>  <!-- порядок важен: зависимости
раньше -->
  <module>demo-rest</module>
</modules>
```

- Maven Reactor сам строит граф зависимостей и соблюдает порядок сборки

```
.\mvnw spring-boot:run # Windows
mvn spring-boot:run # Linux/MacOS
```



Интерфейс как контракт

```
@Tag(name = "Authors", description = "Управление авторами")
@RequestMapping(value = "/api/authors", produces =
APPLICATION_JSON_VALUE)
public interface AuthorApi {

    @Operation(summary = "Получить автора по ID")
    @ApiResponse(responseCode = "200", description = "Автор найден")
    @ApiResponse(responseCode = "404", description = "Автор не
найден")
    @GetMapping("/{id}")
    EntityModel<AuthorResponse> getAuthorById(@PathVariable Long id);

    @Operation(summary = "Создать автора")
    @ApiResponse(responseCode = "201", description = "Автор создан")
    @PostMapping(consumes = APPLICATION_JSON_VALUE)
    ResponseEntity<EntityModel<AuthorResponse>> createAuthor(
        @Valid @RequestBody AuthorRequest request);
}
```

URL, HTTP-метод, коды
ответов, документация,
валидация - всё в
интерфейсе. Один раз.



Реализация контракта

```
@RestController
public class AuthorController implements AuthorApi {
    // HET @RequestMapping – унаследован из AuthorApi
    // HET @GetMapping – унаследован из AuthorApi
    // HET @Valid – унаследован из AuthorApi

    @Override
    public EntityModel<AuthorResponse> getAuthorById(Long id) {
        // Только бизнес-логика:
        return authorModelAssembler.toModel(authorService.findById(id));
    }

    @Override
    public ResponseEntity<EntityModel<AuthorResponse>> createAuthor(AuthorRequest request)
    {
        AuthorResponse created = authorService.create(request);
        EntityModel<AuthorResponse> model = authorModelAssembler.toModel(created);
        return ResponseEntity
            .created(model.getRequiredLink("self").toUri())
            .body(model);
    }
}
```



Request DTO

```
public record AuthorRequest(  
    @NotBlank(message = "Имя не может быть пустым")  
    @Size(max = 100)  
    String firstName,  
  
    @NotBlank(message = "Фамилия не может быть пустой")  
    @Size(max = 100)  
    String lastName,  
  
    @Email(message = "Некорректный формат email")  
    String email,  
  
    @Past(message = "Дата рождения должна быть в прошлом")  
    LocalDate birthDate,  
  
    @Size(max = 2000)  
    String bio  
) {}
```

- records идеальны для Request - неизменяемые, компактные, никакого Lombok...
- Аннотации срабатывают автоматически при наличии @Valid в методе контракта. Без @Valid - аннотации декоративны, проверок не будет



PUT vs PATCH

- Один AuthorRequest не может выразить оба смысла

```
// POST / PUT - поля обязательны, @NotBlank гарантирует
public record AuthorRequest(
    @NotBlank String firstName,    // обязательно
    @NotBlank String lastName,    // обязательно
    @Email      String email,      // необязательно, но при PUT
    // затрётся null
    @Past       LocalDate birthDate
) {}

// PATCH - все поля необязательны, null = «не трогать» на беке
public record PatchAuthorRequest(
    @Size(max = 100) String firstName,    // null = не менять
    @Size(max = 100) String lastName,    // null = не менять
    @Email            String email,       // null = не менять
    @Past             LocalDate birthDate // null = не менять
) {}
```



PUT vs PATCH

- Реализация PATCH в сервисе:

```
String newFirstName = request.firstName() != null
    ? request.firstName()           // пришло новое значение -
берём его
    : existing.getFirstName();      // не пришло - оставляем
старое
```



Response DTO – это классы!

RepresentationModel<T> - базовый класс Spring HATEOAS. Добавляет хранилище _links к ответу

```
@Getter
@Builder
@EqualsAndHashCode(callSuper = false)    // _links не участвуют в equals
@JsonInclude(JsonInclude.Include.NON_NULL) // null-поля не видны в JSON
@Relation(collectionRelation = "authors", itemRelation = "author")
@Schema(description = "Информация об авторе")
public class AuthorResponse extends RepresentationModel<AuthorResponse> {
    @Schema(description = "Уникальный идентификатор", example = "1")
    private final Long id;
    @Schema(description = "Имя автора", example = "Лев")
    private final String firstName;
    @Schema(description = "Фамилия автора", example = "Толстой")
    private final String lastName;
    private final String fullName;    // вычисляемое: firstName + " " +
    lastName
    private final Integer booksCount;
    // ...
}
```



Кастомная валидация

Стандартных аннотаций не хватает? Создаём свою!

```
@Documented
@Constraint(validatedBy = IsbnValidator.class)
@Target({ElementType.FIELD, ElementType.PARAMETER})
@Retention(RetentionPolicy.RUNTIME)
public @interface ValidIsbn {
    String message() default "Некорректный ISBN. Допустимы ISBN-10 и ISBN-13";
    Class<?>[] groups() default {};
    Class<? extends Payload>[] payload() default {};
}
```



Кастомная валидация

Используем:

```
@NotBlank @ValidIsbn String  
isbn
```



Обработка ошибок

Возвращать { "error": "not found" } - непрофессионально. Есть стандарт - RFC 7807 Problem Details!

```
@JsonInclude(JsonInclude.Include.NON_NULL)
public record ErrorResponse(
    int status,           // 404
    String type,          // "https://api.example.com/problems/resource-
not-found"
    String title,         // "Ресурс не найден"
    String detail,        // "Author with id=99 not found"
    String instance,      // "/api/authors/99"
    Instant timestamp,    // "2026-03-10T15:30:00Z"
    List<FieldError> fieldErrors // только для 400, иначе null
) {
    public record FieldError(String field, Object rejectedValue, String
message) {}
}
```

2024. Технологии разработки ПО.



Обработка ошибок

Пример ответа 400:

```
{
  "status": 400,
  "title": "Ошибка валидации",
  "instance": "/api/books",
  "fieldErrors": [
    { "field": "isbn", "rejectedValue": "123", "message": "Некорректный ISBN" },
    { "field": "title", "rejectedValue": "", "message": "Название не может быть пустым" }
  ]
}
```



@ControllerAdvice - централизованная обработка ошибок

```
@ControllerAdvice // перехватывает исключения из всех контроллеров
public class GlobalExceptionHandler {

    private static final String BASE = "https://api.example.com/problems/";

    @ExceptionHandler(ResourceNotFoundException.class)
    public ResponseEntity<ErrorResponse> handleNotFound(
        ResourceNotFoundException ex, HttpServletRequest req) {
        return ResponseEntity.status(NOT_FOUND).body(
            new ErrorResponse(404, BASE + "resource-not-found",
                "Ресурс не найден", ex.getMessage(), req.getRequestURI(),
                Instant.now(), null));
    }
}
```



@ControllerAdvice - централизованная обработка ошибок

```
@ExceptionHandler(MethodArgumentNotValidException.class)
public ResponseEntity<ErrorResponse> handleValidation(
    MethodArgumentNotValidException ex, HttpServletRequest req) {

    List<ErrorResponse.FieldError> errors = ex.getBindingResult()
        .getFieldErrors().stream()
        .map(fe -> new ErrorResponse.FieldError(
            fe.getField(), fe.getRejectedValue(), fe.getDefaultMessage()))
        .toList();

    return ResponseEntity.status(BAD_REQUEST).body(
        new ErrorResponse(400, BASE + "validation-error",
            "Ошибка валидации", null, req.getRequestURI(),
            Instant.now(), errors));
}
```



@ControllerAdvice - централизованная обработка ошибок

```
@ExceptionHandler(Exception.class) // ловит всё остальное
public ResponseEntity<ErrorResponse> handleAll(Exception ex,
HttpServletRequest req) {
    // log.error("Unexpected error", ex); логируем, не теряй стектрейс
    return ResponseEntity.status(INTERNAL_SERVER_ERROR).body(
        new ErrorResponse(500, BASE + "internal-error", "Внутренняя
ошибка",
                        "Обратитесь к поддержке", req.getRequestURI(), Instant.now(),
null));
}
```

Контроллер никогда не обрабатывает ошибки сам - он только бросает исключения!



HATEOAS. Уровни зрелости REST

Уровень	Что умеет	Пример
0	Один URL, всё через POST	POST /api { "action": "getAuthor", "id": 1 }
1	Разные URL на ресурсы	GET /authors/1
2	HTTP-глаголы + коды ответов	DELETE /authors/1 → 204 No Content
3	Ссылки в ответе (HATEOAS)	ответ содержит _links



Уровень 1

```
{ "id": 1, "firstName":  
  "Лев" }
```



Уровень 3

```
{
  "id": 1,
  "firstName": "Лев",
  "_links": {
    "self": { "href":
"http://localhost:8080/api/authors/1" },
    "books": { "href": "http://localhost:8080/api/books?
authorId=1&page=0&size=20" },
    "collection": { "href":
"http://localhost:8080/api/authors?page=0&size=20" }
  }
}
```

Клиент знает наизусть **один** URL. Все остальные получает из `_links`.



Три ключевых класса Spring HATEOAS

- `RepresentationModel<T>` - базовый, добавляет хранилище `_links`

```
public class AuthorResponse extends  
RepresentationModel<AuthorResponse> { ... }
```



Три ключевых класса Spring HATEOAS

- EntityModel<T> - обёртка для одного объекта + ссылки

```
EntityModel.of(author,  
    link("self", "/api/authors/1"),  
    link("books", "/api/books?  
authorId=1"),  
    link("collection", "/api/authors")  
)
```



Три ключевых класса Spring HATEOAS

- PagedModel<EntityModel<T>> - обёртка для страницы объектов

```
{
  "_embedded": { "authors": [ {...}, {...} ] },
  "_links": {
    "self": { "href": "...?page=1&size=20" },
    "first": { "href": "...?page=0&size=20" },
    "prev": { "href": "...?page=0&size=20" },
    "next": { "href": "...?page=2&size=20" },
    "last": { "href": "...?page=4&size=20" }
  },
  "page": { "size": 20, "totalElements": 100, "totalPages": 5,
  "number": 1 }
}
```



Паттерн Assembler

- Без Assembler код добавления `\_links` дублируется везде. С Assembler - одно место:

```
@Component
public class AuthorModelAssembler
    implements RepresentationModelAssembler<AuthorResponse,
EntityModel<AuthorResponse>> {
    @Override
    public EntityModel<AuthorResponse> toModel(AuthorResponse author) {
        return EntityModel.of(author,

            // self: GET /api/authors/{id}
            linkTo(methodOn(AuthorController.class)
                .getAuthorById(author.getId()))
                .withSelfRel(),
```



Паттерн Assembler

- Без Assembler код добавления `\_links` дублируется везде. С Assembler - одно место:

```
...  
// books: GET /api/books?authorId={id}  
linkTo(methodOn(BookController.class)  
    .getAllBooks(author.getId(), null, null, null, 0, 20))  
    .withRel("books"),  
  
// collection: GET /api/authors  
linkTo(methodOn(AuthorController.class)  
    .getAllAuthors(0, 20))  
    .withRel("collection")  
);  
}  
}
```



Паттерн Assembler

`linkTo(methodOn(...))` - не вызов метода! Spring HATEOAS создаёт прокси, читает аннотации (`@GetMapping("/{id}")`), и строит URL. Переименуйте маппинг в контракте - все ссылки обновятся автоматически!

```
...  
// books: GET /api/books?authorId={id}  
linkTo(methodOn(BookController.class)  
    .getAllBooks(author.getId(), null, null, null, 0,  
    20))  
    .withRel("books"),
```



@Relation

```
// Без @Relation – Spring назвал бы коллекцию "authorResponseList":  
@Relation(collectionRelation = "authors", itemRelation = "author")  
public class AuthorResponse extends RepresentationModel<AuthorResponse>  
{ ... }
```



Пагинация

Почему не используем Spring Data Page<T> напрямую в контракте? Потому что тогда контракт зависел бы от Spring Data - а это лишняя зависимость для клиентов.

```
// В контракте простой record (никаких зависимостей Spring Data)
public record PagedResponse<T>(
    List<T> content, int pageNumber, int pageSize,
    long totalElements, int totalPages, boolean last
) {}
```



Пагинация

В контроллере - конвертация в тип, который понимает Spring HATEOAS

```
public PagedModel<EntityModel<AuthorResponse>> getAllAuthors(int page, int
size) {
    // Сервис возвращает наш простой PagedResponse
    PagedResponse<AuthorResponse> paged = authorService.findAll(page, size);
    // Оборачиваем в Spring Data Page<T>
    Page<AuthorResponse> springPage = new PageImpl<>(
        paged.content(),
        PageRequest.of(paged.pageNumber(), paged.pageSize()),
        paged.totalElements()
    );
    // PagedResourcesAssembler строит PagedModel и добавляет ссылки навигации
    return pagedAuthorsAssembler.toModel(springPage, authorModelAssembler);
}
```



OpenAPI

Документация прямо в коде:

```
@Operation(  
    summary = "Создать нового автора",  
    description = "Создаёт автора и возвращает его с HATEOAS-ссылками. " +  
                  "Возвращает Location-заголовок с URL нового ресурса."  
)  
@ApiResponse(responseCode = "201", description = "Автор успешно создан",  
    content = @Content(schema = @Schema(implementation = AuthorResponse.class)))  
@ApiResponse(responseCode = "400", description = "Ошибка валидации",  
    content = @Content(schema = @Schema(implementation = ErrorResponse.class)))  
@PostMapping(consumes = APPLICATION_JSON_VALUE)  
ResponseEntity<EntityModel<AuthorResponse>> createAuthor(  
    @io.swagger.v3.oas.annotations.parameters.RequestBody(  
        description = "Данные нового автора",  
        content = @Content(schema = @Schema(implementation =  
AuthorRequest.class))  
    )  
    @Valid @RequestBody AuthorRequest request);
```



Будут вопросы? Не стесняйтесь!

